

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C ou en OCaml suivant l'exercice. Dans le cas du C, on suppose que les bibliothèques standards usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veuillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Analyse d'un algorithme

« Le problème du drapeau hollandais est un problème de programmation, présenté par Edsger Dijkstra, qui consiste à réorganiser une collection d'éléments identifiés par leur couleur, sachant que seules trois couleurs sont présentes (par exemple, rouge, blanc, bleu, dans le cas du drapeau des Pays-Bas). Étant donné un nombre quelconque de balles rouges, blanches et bleues alignées dans n'importe quel ordre, le problème est à les réarranger dans le bon ordre : les bleues d'abord, puis les blanches, puis les rouges. »

(Wikipedia)

On suppose déjà écrite la fonction `echange` de prototype `void echange(int tab[], int i, int j)` qui échange les éléments d'indice `i` et `j` dans le tableau `tab` et on considère dans la suite que cet échange s'effectue en temps constant. On donne ci-dessous une implémentation de l'algorithme du drapeau hollandais en langage C permettant de réarranger les éléments d'un tableau ne contenant les trois valeurs entières 1, 2 et 3 :

```
1 // Prend en entrée un tableau (ne contenant que les valeurs 1, 2 et 3)
2 // Ne renvoie rien
3 // Réarrange les éléments du tableau de façon à avoir les 1 puis les 2 et les 3
4 void drapeau_hollandais(int tab[], int taille)
5 {
6     int i1 = 0;
7     int i2 = taille - 1;
8     int i3 = taille - 1;
9     while (i1 <= i2){
10         if (tab[i1] == 1){
11             i1 = i1 + 1;
12         }
13         else{
14             if (tab[i1] == 2){
15                 echange(tab, i1, i2);
16                 i2 = i2 - 1;
17             }
18             else{
19                 echange(tab, i1, i2);
20                 echange(tab, i2, i3);
21                 i3 = i3 - 1;
22                 i2 = i2 - 1;
23             }
24         }
25     }
26 }
```

- Q1**– Faire fonctionner cet algorithme sur le tableau `tab` = {1, 3, 2, 2, 3, 1}, et donner le contenu de `tab` ainsi que celui des variables `i1`, `i2`, `i3` lors du déroulement de l'algorithme en recopiant et complétant le tableau suivant :

	<code>tab</code>	<code>i1</code>	<code>i2</code>	<code>i3</code>
Initialisation	{1, 3, 2, 2, 3, 1}	0	5	5
Etape 1	{1, 3, 2, 2, 3, 1}	1	5	5
Etape 2	{1, 1, 2, 2, 3, 3}	1	4	4
Etape 3	{1, 1, 2, 2, 3, 3}	2	4	4
Etape 4	{1, 1, 3, 2, 2, 3}	2	3	4
Etape 5	{1, 1, 2, 2, 3, 3}	2	2	3
Etape 6	{1, 1, 2, 2, 3, 3}	2	1	3

- Q2**– Prouver la terminaison de cet algorithme.

Montrons que $i2-i1$ est un variant :

(H1) $i2-i1$ est entier comme différence de deux entiers

(H2) $i2-i1$ est positif avant d'entrer dans la boucle et reste positif par condition d'entrée dans la boucle `while`

(H3) $i2-i1$ décroît à chaque tour de boucle car soit `i1` augmente (si `tab[i1]==1`) soit `i2` diminue (si `tab[i1]==2` ou `tab[i1]==3`).

Donc $i2-i1$ est bien un variant et donc la fonction termine.

- Q3**– Prouver la correction de cet algorithme.

⊗ *Indication : on pourra noter n la taille du tableau et :*

- P_1 la tranche du tableau `tab` comprise entre les indices 0 et `i1` (*exclu*)
- P_2 la portion du tableau `tab` comprise entre les indices `i2` et `i3` (*exclu*)
- P_3 la portion du tableau `tab` comprise entre les indices `i3` et $n-1$ (*exclu*)

Et prouver l'invariant suivant : « P_1 ne contient que des 1, P_2 que des 2 et P_3 que des 3 ».

Avec les notations proposées pour P_1 , P_2 et P_3 , montrons l'invariant = « P_1 ne contient que des 1, P_2 que des 2 et P_3 que des 3 »

- Initialisation : avant d'entrer dans la boucle, $i_1 = 0$, $i_2 = n-1$, $i_3 = n-1$ donc P_1 , P_2 et P_3 sont vides et donc I est vraie.
- Conservation : On suppose I vérifié et on montre qu'il reste vrai au tour de boucle suivant, on distingue alors 3 cas suivant la première valeur non encore triée c'est à dire `tab[i1]` :
 - Si `tab[i1]` vaut 1, alors `i1` augmente de 1 donc un 1 est ajouté à P_1 qui puisque I est supposé vérifié en entrant dans la boucle ne contenait que des 1. Ni P_2 ni P_3 ne sont modifiés et donc dans ce cas I reste vrai.
 - Si `tab[i1]` vaut 2, alors ce 2 est placé à l'indice `i2` qui est décrémenté. Cela revient donc à ajouter un 2 à P_2 qui par hypothèse ne contenait que des 2. Ni P_1 , ni P_3 ne sont modifiés et donc I reste vrai.
 - Sinon `tab[i1]` vaut nécessairement 3, alors ce 3 est placé à l'indice `i3`, La partie P_2 est décalée d'un rang à gauche (elle garde la même longueur), i_2 et i_3 sont décrémentés. Un 3 étant ajouté à P_3 qui par hypothèse ne contenait que des 3, l'invariant est préservé.

On en conclut que la fonction est totalement correcte (elle termine et elle est correcte)

- Q4**– Donner, en la justifiant brièvement, la complexité de l'algorithme du drapeau hollandais.

En notant n la taille du tableau, la boucle `while` s'exécute n fois et comme elle ne contient que des opérations élémentaires, la complexité de l'algorithme est un $O(n)$.

- Q5–** On rappelle que l'algorithme du tri par sélection consiste à rechercher pour $i=0 \dots i=n-1$ le minimum du sous tableau `tab[i] ... tab[n-1]` et à l'échanger avec l'élément d'indice i . Donner une implémentation de cet algorithme de tri en langage C sous la forme d'une fonction de signature :
void tri_selection(int tab[], int size) qui prend en argument un tableau d'entiers et sa longueur et trie en place de tableau. On pourra *éventuellement* utiliser une fonction annexe de recherche de minimum dont on précisera la spécification.

```

1 // Renvoie l'indice du minimum des éléments de tab depuis l'indice ind
2 int min_depuis(int tab[], int taille, int ind)
3 {
4     assert(ind < taille && ind >= 0);
5     int vmin = tab[ind];
6     int imin = ind;
7     for (int i = ind; i < taille; i++)
8     {
9         if (tab[i] < vmin)
10        {
11            imin = i;
12            vmin = tab[i];
13        }
14    }
15    return imin;
16 }
17
18 // Tri en place un tableau par sélection
19 void tri_selection(int tab[], int size)
20 {
21     int im;
22     for (int i = 0; i < size; i++)
23     {
24         im = min_depuis(tab, size, i);
25         echange(tab, i, im, size);
26     }
27 }

```

- Q6–** Rappeler, en la justifiant brièvement, la complexité de l'algorithme du tri par sélection.

Le tri par sélection a une complexité quadratique en effet la fonction de recherche du minimum est de complexité linéaire en la taille du tableau et elle est appelée à chaque itération de la boucle `for`.

- Q7–** On a mesuré qu'en utilisant l'algorithme du tri par sélection un ordinateur trie une liste de dix million d'éléments en 5 secondes. Quel est le temps prévisible approximatif pour trier une liste contenant un milliard d'éléments ?

Le nombre d'éléments est multiplié par 100 et l'algorithme ayant une complexité quadratique, le temps prévisible sera multiplié par $100^2 = 10\,000$. Donc l'exécution prendra environ 50 000 secondes (environ 14 heures).

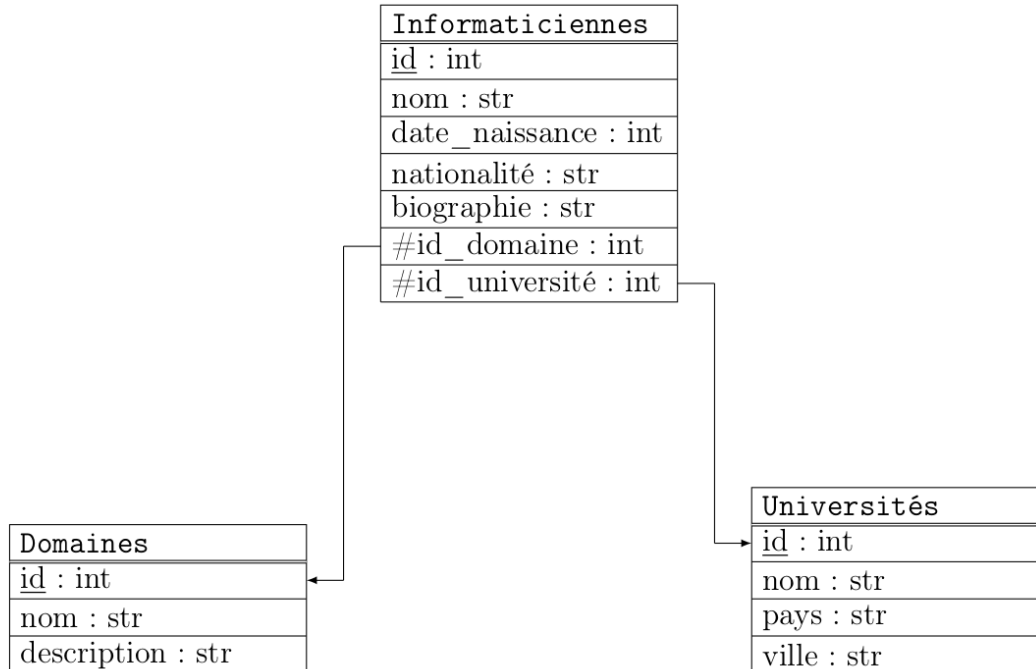
- Q8–** On a mesuré qu'en utilisant l'algorithme du drapeau hollandais un ordinateur trie une liste de dix million d'éléments en 0.2 secondes. Quel est le temps prévisible approximatif pour un trier une liste contenant un milliard d'éléments ?

Le nombre d'éléments est multiplié par 100 et l'algorithme ayant une complexité linéaire, le temps prévisible sera multiplié lui aussi par 100. Donc l'exécution prendra environ 20 secondes.

❑ **Exercice 2** : *Base de données et informaticiennes célèbres*

🎓 CAPES L3 – Sujet zéro

On souhaite modéliser une base de données sur des informaticiennes ayant marqué l'histoire de l'informatique (exemples : Ada Lovelace, Grace Hopper, Frances Alln, Radia Perlman, ...). Cette base de données repose sur le schéma relationnel suivant



La table **Domaines** contient les différents domaines de recherche des universités.

Q9– Indiquer pour chaque table, la clé primaire et la ou les éventuelles clés étrangères.

La clé primaire de **Domaines** est **id**, celle de **Universités** porte aussi le nom **id** et celle de **Informaticiennes** porte aussi le nom **id**. La table **Informaticiennes** possède une clé étrangère **id_domaines** qui référence la clé primaire de **Domaines** et une clé étrangère **id_université** qui référence la clé primaire de **Universités**.

Q10– Ecrire une requête SQL permettant d'afficher dans l'ordre alphabétique le nom des informaticiennes françaises.

```

SELECT nom
FROM Informaticiennes
WHERE nationalite="Française"
ORDER BY nom ASC
  
```

Q11– Ecrire une requête SQL permettant d'afficher le nombre d'informaticiennes américaines.

```

SELECT COUNT(*)
FROM Informaticiennes
WHERE nationalite="Américaine"
  
```

Q12– Ecrire une requête SQL affichant le nom de chaque informaticienne, le nom de son domaine de recherche et le nom de son université.

```

SELECT Informaticiennes.nom, Domaines.nom, Universites.nom
FROM Informaticiennes
JOIN Domaines ON Informaticiennes.id_domaine = Domaines.id
JOIN Universites ON Informaticiennes.id_universite = Universites.id
  
```

- Q13–** Ecrire une requête SQL affichant le nombre d'informaticiennes par nationalité en triant le résultat par nombre décroissant.

```
SELECT nationalite, COUNT(*) as nb
FROM Informaticiennes
GROUP BY nationalite
ORDER BY nb DESC
```

□ **Exercice 3 : L'accès aux enfers**

Dans la mythologie grecque, l'accès aux Enfers est gardé par le Cerbère, un terrible loup à trois têtes. Celui-ci se trouve devant trois couloirs qui, soit permettent de rejoindre le monde des vivants, soit conduisent directement aux Enfers.

Lorsque Cerbère accueille un nouvel arrivant, il est contraint de lui dire la vérité. Par la suite, il peut mentir ou dire la vérité à sa guise mais il respecte toujours les règles qu'il s'est fixées.

Après avoir bu la coupe de ciguë, Socrate se retrouve face à Cerbère. Celui-ci, honoré de rencontrer le grand philosophe, veut lui offrir une chance d'éviter la damnation éternelle. Il lui dit alors : « *Je vais t'indiquer un des couloirs qui mène au monde des vivants mais, pour mettre à l'épreuve ta grande sagesse, j'énoncerai trois indications logiques qui seront, soit toutes vraies, soit toutes fausses et tu en déduiras le couloir que tu devras suivre* ».

Nous noterons I_1 , I_2 et I_3 les propositions associées aux indications de la première, la deuxième et la troisième tête de Cerbère.

- Q14–** Représenter l'énoncé de Cerbère sous la forme d'une formule du calcul des propositions dépendant de I_1 , I_2 et I_3

$$(I_1 \wedge I_2 \wedge I_3) \vee (\neg I_1 \wedge \neg I_2 \wedge \neg I_3)$$

La première tête dit ensuite : « *Le premier couloir ainsi que le troisième mènent au monde des vivants* ». La deuxième tête dit : « *Si le deuxième couloir mène au monde des vivants, alors le troisième n'y mène pas* ». La troisième tête conclut par : « *Le premier couloir mène au monde des vivants, par contre le deuxième n'y mène pas* ».

Nous noterons, C_1 , C_2 et C_3 les variables propositionnelles correspondant au fait que le premier, le deuxième et le troisième couloir mène au monde des vivants.

- Q15–** Exprimer I_1 , I_2 et I_3 sous la forme du calcul des propositions dépendant de C_1 , C_2 et C_3

- $I_1 : C_1 \wedge C_3$
- $I_2 : C_2 \rightarrow \neg C_3$
- $I_3 : C_1 \wedge \neg C_2$

- Q16–** En utilisant le calcul des propositions (résolution avec les tables de vérité), déterminer le couloir que Socrate doit suivre pour rejoindre le monde des vivants.

C_1	C_2	C_3	I_1	I_2	I_3	$(I_1 \wedge I_2 \wedge I_3) \vee (\neg I_1 \wedge \neg I_2 \wedge \neg I_3)$
0	0	0	0	1	0	0
0	0	1	0	1	0	0
0	1	0	0	1	0	0
0	1	1	0	0	0	1
1	0	0	0	1	1	0
1	0	1	1	1	1	1
1	1	0	0	1	0	0
1	1	1	1	0	0	0

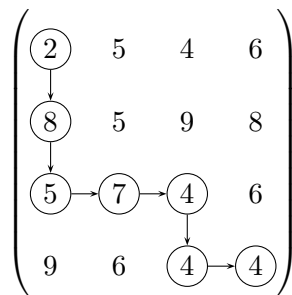
Donc comme I_1 , I_2 et I_3 sont soit toutes vraies soit toutes fausses, seules deux lignes du tableau conviennent et dans les deux cas, C_3 est vraie, donc pour être sûr de rejoindre le monde des vivants, Socrate doit prendre le couloir 3.

- Q17–** En admettant que Cerbère ait menti en donnant les trois indications, Socrate pouvait-il suivre d'autres couloirs ? Si oui, le ou lesquels ?

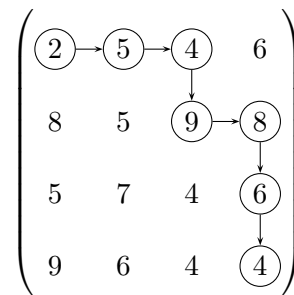
Si on suppose que Cerbère a menti alors on se trouve sur la ligne où I_1, I_2 et I_3 sont fausses et donc deux couloirs permettent de rejoindre le monde des vivants le couloir 2 et le couloir 3.

□ **Exercice 4 :** *Traversée maximale d'une matrice carrée*

On considère une matrice à n lignes et n colonnes notée M à coefficients dans $\mathbb{N}$, et dont le coefficient situé à la ligne i et à la colonne j avec $0 \leq i \leq n-1$ et $0 \leq j \leq n-1$ sera noté $M_{i,j}$. On s'intéresse aux chemins depuis la première valeur en haut à gauche ($M_{0,0}$) jusqu'à la dernière en bas à droite ($M_{n-1,n-1}$) qui n'utilisent que les déplacements vers la droite ($\rightarrow$) ou vers le bas ($\downarrow$). Voici deux exemples de tels chemins dans une même matrice A de 4 lignes et 4 colonnes :



Un exemple C_1 de chemin dans A

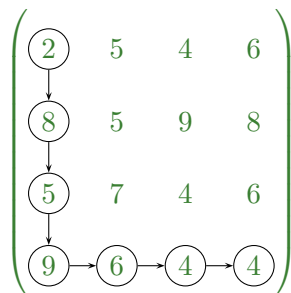


Un exemple C_2 de chemin dans A

On appelle *somme d'un chemin* dans une matrice M , la somme des coefficients $M_{i,j}$ rencontrés en suivant ce chemin. Dans les exemples ci-dessus, le chemin C_1 a une somme de 34 et le chemin C_2 a une somme de 38. Le but du problème est d'étudier divers algorithmes qui cherchent à déterminer un chemin dont la somme est maximale.

On propose l'algorithme $\mathcal{A}_0$ suivant afin de résoudre ce problème : si les deux directions bas ou droite sont possibles on choisit celle qui mène vers le coefficient de matrice le plus élevé, en cas d'égalité, on choisit (arbitrairement) la direction bas. Dans l'exemple ci-dessus à partir de $M_{0,0} = 2$ on peut aller vers à droite vers $M_{0,1} = 5$ ou en bas vers $M_{1,0} = 8$ et on choisit donc d'aller en bas puisque $8 > 5$. Une fois arrivé en $M_{1,0}$ les deux directions possibles mènent vers des coefficients identiques (5 des deux côtés) et donc, on va vers le bas. Enfin, lorsqu'une seule direction est possible (parce qu'on a atteint la dernière colonne ou la dernière ligne), c'est cette direction qui est choisie.

- Q18–** Poursuivre le déroulement de cet algorithme sur la matrice A donnée en exemple ci-dessus, dessiner le chemin obtenu et donner sa somme.



Le chemin obtenu a une somme de $2 + 8 + 5 + 9 + 6 + 4 + 4 = 38$

- Q19–** Dans quelle famille d'algorithme peut-on classer cet algorithme ? Justifier.

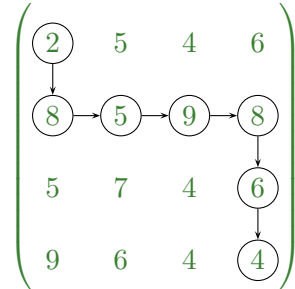
C'est un algorithme glouton car à chaque étape on choisit un maximum *local* sans se préoccuper de l'impact de ce choix sur le maximum global.

- Q20–** Donner la complexité de cet algorithme.

On doit choisir un nombre de directions qui dépend linéairement de la taille de la matrice n , chaque choix compare deux valeurs et donc s'effectue en temps constant. C'est donc un algorithme de complexité linéaire.

- Q21–** Montrer en exhibant un exemple dans le cas $n = 4$ que cet algorithme ne fournit pas toujours un chemin de somme maximale.

On a obtenu sur l'exemple une somme de 38, or dans cette même matrice, un autre chemin donne une somme supérieure :



Le chemin obtenu a une somme de $2 + 8 + 5 + 9 + 8 + 6 + 4 = 46$

En Ocaml, on décide de représenter un déplacement vers la droite par un 0 et un déplacement vers le bas par un 1. Un chemin dans la matrice est alors représenté par une liste de 0 et de 1, par exemple le chemin C_1 donné en exemple est représenté par $[1, 1, 0, 0, 1, 0]$.

- Q22–** Ecrire une fonction `choix : int array array -> int -> int -> int` qui prend en argument une matrice ainsi qu'un numéro de ligne et de colonne et valide (c'est à dire positif et strictement inférieur à la dimension de la matrice) et renvoie le déplacement choisi (0 pour la droite et 1 pour le bas) par l'algorithme naïf $\mathcal{A}_0$.

```
1 let choix matrice i j =
2   let n = Array.length matrice in
3   if i==n-1 then 0 else
4     if j==n-1 then 1 else
5       if matrice.(i+1).(j) >= matrice.(i).(j+1) then 1 else 0
```

- Q23–** Ecrire une fonction `naif : int array array -> int list` qui prend en entrée une matrice d'entiers M et renvoie le chemin fourni par l'algorithme naïf $\mathcal{A}_0$. Les réponses n'utilisant pas les aspects impératifs du langage (boucle et référence) seront valorisées.

```
1 let naif matrice =
2   let n = Array.length matrice in
3   let rec aux i j =
4     if i=n-1 && j=n-1 then [] else
5     let c = choix matrice i j in
6     if c=1 then 1::aux (i+1) j else 0::aux i (j+1) in
7   aux 0 0;;
```

- Q24–** Ecrire une fonction `somme : int array array -> int list -> int` qui prend en entrée une matrice d'entiers M , un chemin dans cette matrice et renvoie la somme de ce chemin. De même que pour la question précédente, les réponses n'utilisant pas les aspects impératifs du langage seront valorisées.

```

1 let somme matrice chemin =
2   let rec aux chemin i j =
3     match chemin with
4     | [] -> 0
5     | d::reste -> if d=0 then matrice.(i).(j+1)+(aux reste i (j+1))
6                   else matrice.(i+1).(j)+(aux reste (i+1) j) in
7   matrice.(0).(0) + aux chemin 0 0;;

```

Q25– Montrer que pour une matrice de taille n , un chemin de $M_{0,0}$ à $M_{n-1,n-1}$ est la donnée de $2n - 2$ déplacements et que parmi ces déplacements $n - 1$ sont vers la droite et $n - 1$ sont vers le bas.

On doit effectuer $n - 1$ déplacements vers la droite afin de passer de la colonne 0 à la colonne $n - 1$. De même on effectue $n - 1$ déplacements vers le bas afin de passer de la ligne 0 à la ligne $n - 1$. Ce qui donne un total de $2n - 2$ déplacements.

Q26– En déduire que pour une matrice M de n lignes et n colonnes, il existe $\binom{2n-2}{n-1}$ chemins possibles de $M_{0,0}$ vers $M_{n-1,n-1}$.

D'après la question précédente, un chemin est une liste de $2n - 2$ valeurs contenant $n - 1$ fois 0 et $n - 1$ fois 1. Choisir un chemin revient donc à choisir l'emplacement des $n - 1$ valeurs 1 parmi les $2n - 2$ emplacements disponibles. Donc il y a en tout $\binom{2n-2}{n-1}$ chemins.

Q27– En déduire la complexité de l'algorithme par force brute qui testerait le coût de tous les chemins possibles afin de déterminer le chemin optimal. On pourra utiliser le résultat suivant : $\binom{2n-2}{n-1}$ est un $\mathcal{O}(4^n)$.

Comme pour chaque chemin, le calcul de la somme est un $\mathcal{O}(n)$, on obtient finalement une complexité $\mathcal{O}(4^n n)$, c'est à dire une complexité exponentielle. L'algorithme est donc inutilisable en pratique pour de grandes valeurs de n .

On veut maintenant utiliser une méthode par programmation dynamique pour résoudre ce problème. On note, pour tout $i \in \llbracket 0; n - 1 \rrbracket$ et $j \in \llbracket 0; n - 1 \rrbracket$ $S_{i,j}$ la somme maximale d'un chemin de $M_{i,j}$ vers $M_{n-1,n-1}$.

Q28– Justifier rapidement que $S_{n-1,n-1} = 0$.

On se trouve déjà sur $M_{n-1,n-1}$, le seul chemin est donc le chemin vide de somme nulle et $S_{n-1,n-1} = 0$.

Q29– Justifier que pour tout $0 \leq i < n - 1$, $S_{i,n-1} = M_{i+1,n-1} + S_{i+1,n-1}$ et que pour tout $0 \leq j < n - 1$, $S_{n-1,j} = M_{n-1,j+1} + S_{n-1,j+1}$.

Pour tout $0 \leq i < n - 1$, le seul mouvement autorisé depuis $M_{i,n-1}$ est d'aller en bas car on a déjà atteint la dernière colonne, donc la somme maximale d'un chemin depuis $S_{i,n-1}$ est celle obtenue depuis $M_{i+1,n-1}$ auquel on ajoute la somme maximale obtenue depuis $M_{i+1,n-1}$ c'est à dire $S_{i+1,n-1}$, on a donc bien $S_{i,n-1} = M_{i+1,n-1} + S_{i+1,n-1}$. On obtient la seconde relation par un raisonnement similaire lorsque la dernière ligne est atteinte.

Q30– Montrer que pour $0 \leq i < n - 1$ et $0 \leq j < n - 1$, $S_{i,j} = \max \{M_{i+1,j} + S_{i+1,j}, M_{i,j+1} + S_{i,j+1}\}$.

A partir de $M_{i,j}$, on peut aller en bas et dans ce cas la somme maximale atteinte sera $M_{i+1,j} + S_{i+1,j}$ ou à droite et dans ce cas la somme maximale atteinte sera $M_{i,j+1} + S_{i,j+1}$. On prend le maximum de ces possibilités, ce qui donne bien l'égalité ci-dessus.

Q31– Ecrire une fonction *réursive, dynamique* : `int array array -> int` qui calcule la solution du problème (à savoir $S_{0,0}$) en utilisant les relations de récurrences écrites aux questions précédentes.


```

1  let dynamique matrice =
2    let n = Array.length matrice in
3    let rec aux i j =
4      if i=n-1 && j=n-1 then 0 else
5        if i=n-1 then matrice.(i).(j) + aux i (j+1) else
6          if j=n-1 then matrice.(i).(j) + aux (i+1) j else
7            let bas =aux (i+1) j in
8            let droite = aux i (j+1) in
9            max (matrice.(i+1).(j) + bas) (matrice.(i).(j+1) + droite) in
10   matrice.(0).(0) + aux 0 0;;

```

Q32– Faire un schéma des premiers appels récursifs de la fonction précédente, quel phénomène constate-t-on ?

On constate un chevauchement des appels récursifs, c'est à dire que plusieurs appels récursifs avec les mêmes paramètres d'appel.

Q33– La mémoïsation est une méthode permettant de palier le problème mis en évidence à la question précédente, expliquer brièvement en quoi cela consiste (on ne demande pas de programmer cette méthode mais simplement d'en expliciter le principe).

La mémoïsation consiste à stocker les résultats déjà calculés afin d'en disposer rapidement si le même appel intervient de nouveau. La structure de données adaptée est une table de hachage, les clés sont les paramètres d'appel et les valeurs associées le résultat de la fonction. Ici on peut simplement utiliser une matrice de dimension $n \times n$ car les paramètres d'appel sont des entiers i, j appartenant à $\llbracket 0; n \rrbracket$.

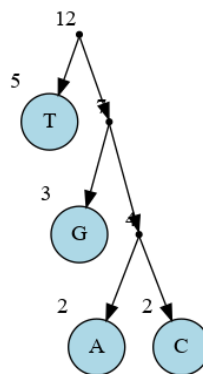
Q34– Déterminer la complexité de cette la méthode par programmation dynamique si on suppose qu'on utilise la mémoïsation.

La complexité est quadratique en la taille n de la matrice, en effet on calcule les valeurs de la matrice des S_{ij} en $O(1)$ et chaque terme est calculé en une seule fois.

□ Exercice 5 : Séquence génétique

On s'intéresse dans cet exercice à la compression de chaînes de caractères représentant des séquences génétiques codées sur l'alphabet $\{A, C, G, T\}$. Et on souhaite compresser la séquence $S=ATGTGATGTCCT$. On supposera qu'initialement la séquence est codée en ASCII et que donc chacun des caractères occupe 1 octet (donc 8 bits).

Q35– Construire l'arbre de Huffman associé à la compression de S .



Q36– Donner les codes obtenus pour chacun des quatre caractères A, C, G et T. En déduire le taux de compression de l'algorithme, sans tenir compte de la taille de l'arbre (on pourra conserver une écriture fractionnaire).

- Code pour T = 0
- Code pour G = 10
- Code pour A = 110
- Code pour C = 111

Comme indiqué sur l'arbre T apparaît 5 fois, G 3 fois et A et C chacun deux fois. La taille finale du code est donc : $5 \times 1 + 3 \times 2 + 2 \times 3 + 2 \times 3 = 23$ bits, alors le code initial contenait 12 caractères codés sur 8 bits chacun donc 96 bits, le taux de compression est donc de $\frac{23}{96} \simeq 24\%$.

On rappelle que le principe de la compression LZW est d'attribuer un code aux préfixes rencontrés lors de la lecture du texte à compresser de façon à disposer d'un code compact si ce préfixe se présente à nouveau. On attribue initialement les codes $A \rightarrow 0$, $C \rightarrow 1$, $G \rightarrow 2$ et $T \rightarrow 3$. Le début de l'algorithme va donc consister à attribuer un nouveau code au premier préfixe non encore codé qui apparaît lors de la lecture du texte. Et donc, ici, on attribue le code 4 au préfixe AT et on émettra le code de A c'est à dire 0.

Q37– Poursuivre le déroulement de cet algorithme en complétant le tableau suivant :

Position dans le texte	Code émis	Nouveau préfixe ajouté
<u>A</u> TGTGATGTCCT	0	AT $\rightarrow$ 4

Position dans le texte	Code émis	Nouveau préfixe ajouté
<u>A</u> TGTGATGTCCT	0	AT $\rightarrow$ 4
A <u>T</u> GTGATGTCCT	3	TG $\rightarrow$ 5
AT <u>G</u> TGATGTCCT	2	GT $\rightarrow$ 6
ATGT <u>G</u> ATGTCCT	5	TGA $\rightarrow$ 7
ATGTGA <u>A</u> TGTCCT	4	ATG $\rightarrow$ 8
ATGTGATG <u>T</u> CCT	6	GTC $\rightarrow$ 9
ATGTGATGT <u>C</u> CT	1	CC $\rightarrow$ 10
ATGTGATGTCT <u>C</u>	1	CT $\rightarrow$ 11
ATGTGATGTCTC <u>T</u>	3	

On obtient donc la suite de codes : [0 ; 3 ; 2 ; 5 ; 4 ; 6 ; 1 ; 1 ; 3]

Q38– Quel est le taux de compression obtenu (la taille d'un code est un octet) ?

12 sur 9

Q39– Décompresser le texte T codé par suite de codes [3 ; 1 ; 4 ; 6 ; 5 ; 2 ; 0 ; 2] sur ce même alphabet en expliquant comment est reconstruit le dictionnaire de décompression.

$T = \text{TCTCTCTCTGAG}$

A présent, on code en base 2 sur deux bits les nucléotides :

Nucléotide	A	C	G	T
Code sur 2 bits	00	01	10	11

Et on propose de représenter un brin d'ADN par une suite d'entiers codés sur 8 bits, les deux premiers bits de l'entier représentent le nucléotide et les 6 bits suivants indiquent le nombre de répétitions. Par exemple

- l'entier 10 s'écrit en binaire sur un octet $(00001010)_2$, les deux premiers bits sont 00 et donc représentent le nucléotide A, les 6 bits suivants donnent le nombre de répétitions $(001010)_2 = (10)_{10}$ et donc cet entier code AAAAAAAAAA.
- l'entier 167 s'écrit en binaire sur un octet $(10100111)_2$, comme 10 (les deux premiers bits) code le nucléotide G et que les 6 bits suivants représentent $(100111)_2 = (39)_{10}$ en décimale, cet entier représente le nucléotide G répété 39 fois.

- l'entier 200 s'écrit en binaire sur un octet $(11001000)_2$ et donc représente 8 répétitions du nucléotide T.

Q40– Que représente l'entier 101 ?

$(101)_{10} = (01100101)_2$ et donc cet entier représente le nucléotide C répété 37 fois.

Q41– Par quel entier peut-on représenter TTTTTTTTTT (10 répétitions de T) ?

On doit représenter le nucléotide T donc les premiers bits sont 11 puis on écrit 10 en base 2 sur les 6 bits restants ce qui donne $(11001010)_2 = (202)_{10}$.

Q42– Avec ce codage, quel est le nombre maximal de répétitions d'un nucléotide que l'on peut coder avec un entier sur 8 bits ?

Le nombre de répétitions est codé sur 6 bits et donc le nombre maximal de répétitions est $(111111)_2 = (63)_{10}$.